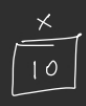
Autoboxing / Unboxing

Data Types

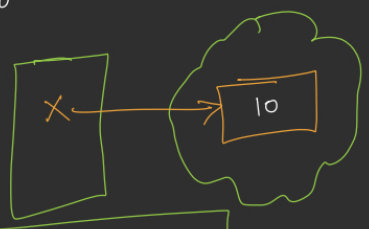
- Primitives
- Non-primitives

Primitives	Non-primitives
int	Integer
long	Long
short	Short
float	Float
double	Double
char	Character
boolean	Boolean

```
int x = 10;
```




```
Integer x = new Integer(10);
```




```
Class Integer {
    int value;
    // Constructors
    // Methods
}
```

Primitives	Wrappers
int	Integer
float	Float
⋮	⋮

① → Legacy.

② → Fast

Autoboxing / Unboxing

int → Integer

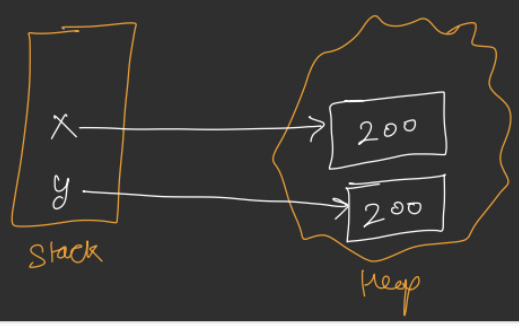
```
Class Integer {
    int value;
    // Constructor
    static Integer valueOf(int x) {
    }
}
```

```
int x = 10;
Integer y = x; // Autoboxing
Integer y = new Integer(x); // Old
Integer y = Integer.valueOf(x); // New
```

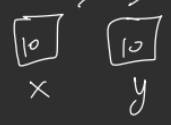
```
Integer x = 10;
int y = x;          // unboxing.

int y = x.intValue();

class Integer {
    int value;
    // Constructors.
    static Integer valueOf(int x){
        // something
    }
    int intValue() {
        return value;
    }
}
```

```
int x = 100;
int y = 100;

(x==y)  // true.

Integer x = 200;
Integer y = 200;
(x==y)  // false.
```

```
Integer x = 200;  // Integer y = 200;
↓
Integer x = Integer.valueOf(200);
↓
x = new Integer(200);
```




```
int x = 10;
int y = 10;
```



(==)
↓
It compares References.

x.equals(y)
↳ true

```
Public final Class Integer {
    Private int value;
    Public Integer (int value) {
        this.value = value;
    }
    Public int intValue () {
        return value;
    }
    Public static Integer valueOF (int x) {
        // Give Object Integer of x
    }
    Public boolean equals ( Integer y) {
        return (this.value == y.value);
    }
}
```

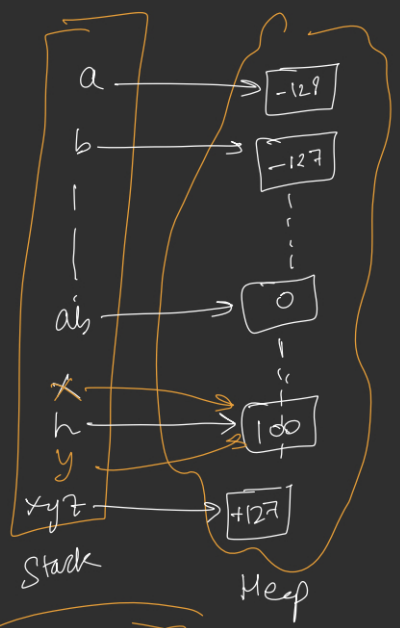
// Conceptual

```
Integer x = 200;
Integer y = 200;
(x == y) // False
x = 100;
y = 100
(x == y) // true
```

Integer

Value of

(-128 to +127)




```
Integer x = 100;
x = Integer.valueOF (100);
Integer y = 100;
y = Integer.valueOf (100);
(x == y)
```

-128 — +127

Abstract Classes

↳ Any class which Cant be instantiated.

POJO Classes

↳ Plain Old Java Object

Student → name, age, rollno, college

```
class Sample {
    int x;
    String y;
    Sample (int x, String y){
    }
    int getx() {
        return x;
    }
    void setx( int x) {
    }
}
```

```
class Student {
}
```

POJO

↳ Hard Core Business logic (X)

→ getter / setter
→ Constructor
→ variable (fields)
→ Build cx.
→ Business logic

POJO

Anemic Model

- getter/setter
- Constructor
- fields

Rich Domain Model

- Business logic

```
class Student {
    String name;
    Student (name) {
    }

    // getters
    // setters
}
```

```
markAttendance() {
}
```